Plan 2 del TP6 para catálogo compras e inventario móvil

Versión 3 del 29 de octubre de 2026, revisada tras el merge de auditoría. Entregar este archivo completo al agente o alumno 2. Reemplaza el Plan 2 anterior. El compañero utiliza TP6_Plan_1_Base_acceso_administracion_y_reportes.md. No es una autorización para crear ahora el frontend ni subir código.

1 Contexto y criterio de alcance

El TP6 exige un frontend Blazor Hybrid integrado con la API ASP.NET del TP5, orientado y probado en Android. Debe demostrar catálogo con stock, carrito, compra y pago simulado, ventas, entradas, control de existencias, reportes PDF consultables y enviables por correo, acceso y roles, auditoría y log de errores. Debe trabajarse entre dos alumnos con Git y justificar las tres primeras formas normales de la base.

La consigna académica prevalece sobre INTRUCCIONES.docx. Se elimina la obligación de ranking de más vendidos, dashboards con conteos especiales, layout exacto de hamburguesa/footer, tabla para todo CRUD, edición de roles ya asignados y borrado/rechazo administrativo de pedidos. No construir un clon completo de Mercado Libre ni agregar pagos reales, logística, vendedores múltiples, wishlist o notificaciones. No perseguir cobertura visual de todos los endpoints antes de completar la compra móvil.

MudBlazor y SweetAlert2 no son exigencias académicas: empezar con Bootstrap local y componentes Razor simples; confirmar acciones destructivas y las que cambian stock, sin popup en cada paso de cantidad. Mantener diseño blanco/gris con acento verde petróleo como preferencia visual, sin convertirlo en requisito docente. El pago simulado sí es obligatorio y queda como pantalla y prueba explícita en este plan.

La revisión inicial usó la copia local 7429d12; se contrastó además main remoto 114bce7, que integra la rama auditoria 711b269 por PR #28. Usar ese contrato actualizado y verificar Controllers, DTOs, OpenAPI y migraciones al iniciar. El proceso local aún puede ejecutar código anterior. No crear endpoints ni cambiar backend/migraciones desde el frontend. Una incompatibilidad se explica al propietario y se propone solución; no falsear cumplimiento. No hacer push, subir código o tocar datos reales por este documento. El propietario incorpora los cambios manualmente. Los dos planes están fuera del repositorio. La estructura autorizada posterior se encuentra en Apinventario.App con carpetas y README; conservarla al generar la plantilla futura, aún pendiente.

2 Referencias y estructura simple en un repositorio

PuntoVenta (https://github.com/federicotrani/PuntoVenta) separa Client, Server y Shared, pero usa WebAssembly; tomar su organización, no su tecnología de cliente ni sesión mock. El ejemplo móvil https://github.com/AgusBuss/tienda-de-negocios- incluye InventarioApp MAUI con WebView.Maui y carpetas Components, Models, Services, Platforms y Resources; sirve de referencia estructural, sin asumir que cumple toda la consigna.

Propuesta: mantener TPN°5-Movil como backend y añadir en el futuro Apinventario.App como carpeta hermana, con una solución agrupadora Apinventario.slnx. No hacen falta cinco proyectos de frontend ni una biblioteca Shared extra para empezar. La app no referencia entidades EF ni accede a SQL: Models contiene DTOs HTTP.

```
raíz/
  TPN°5-Movil/                    API existente solo lectura
  Apinventario.App/
    Components/
      Layout/ Shared/             agente 1
      Pages/
        Acceso/ Administracion/ Auditoria/ Reportes/   agente 1
        Tienda/ Compras/ Inventario/                   agente 2
    Services/
      Infrastructure/ Administracion/                 agente 1
      Comercio/                                       agente 2
    Models/
      Common/ Administracion/                         agente 1
      Comercio/                                       agente 2
    Platforms/ Resources/ wwwroot/                     agente 1
  tests/Apinventario.App.Unit/Agente1 y Agente2/
  docs/tp6/agente-1/ y docs/tp6/agente-2/
```

Agente 1 es único dueño del csproj, MauiProgram, MainPage/App XAML y code behind, Router, imports globales, layout, CSS global, paquetes, configuración Android y solución. B solo edita Tienda, Compras, Inventario, Models/Comercio, Services/Comercio, sus .razor.css, imports locales y sus pruebas/documentación. No añadir otro .git. A integra referencias/registros; B solicita cambios de infraestructura con firma y justificación, no los realiza por su cuenta.

B comienza DTOs, servicios de dominio, pantallas y dobles de desarrollo mientras A prepara host y contratos comunes. Antes de integración real debe recibir ese primer hito compilable. No crear otra sesión/ApiClient para evitar esperar. Enviar a A inventario de rutas, roles y extensión AddComercio. Una sola bandeja de solicitudes para Empleado y Administrador y una sola gestión de productos/proveedores evitan duplicación.

3 Contratos comunes y estilo

Consumir los servicios de A en Services.Infrastructure:

- SessionService: CurrentUser con Id/Rol, IsAuthenticated, caducidad y eventos; login/restauración/logout pertenecen a A. Token en SecureStorage; B no lo lee ni lo persiste directamente.

- ApiClient: SendJsonAsync<T>(HttpMethod,string relativePath,object? body,CancellationToken), SendNoContentAsync, SendMultipartAsync<T>, DownloadAsync. Bearer, serialización y ApiFailure centralizados; error con código, mensaje, validaciones y RetryAfter. No segundo HttpClient sin autorización.

- ApiConfig: BaseUri y ResolveImageUrl. Sin localhost dispersos ni conexión SQL en móvil.

- ConfirmService: ConfirmAsync(titulo,mensaje,confirmar,cancelar). Cancelar no envía petición. Usar para baja/reversión, envío final y aceptación; acciones normales de formulario no requieren popup si no existe riesgo especial.

- FileService: selección de imagen y apertura de archivos de forma nativa; AppErrorLogger para fallos sanitizados sin tokens ni datos privados.

- PageResult<T>: page/pageSize/totalCount/totalPages/hasNext/hasPrevious/items; page desde 1, pageSize 1-100. DTOs de dominio Comercio son de B y fieles al backend.

A concreta firmas antes de integración; B acuerda cualquier ajuste. Registrar clientes mediante AddComercio, llamado por A en MauiProgram. Tests en carpeta Agente2; A crea csproj de pruebas compartido.

Paleta común: #F6F7F9 fondo, #FFFFFF superficies, #20252B texto, #59636F secundario, #DDE2E7 bordes, #0F766E acento y #B42318 peligro. Fuente de sistema 16 px, títulos 22-24, márgenes 16 y escala 4/8/12/16/24/32, radios suaves 10-12. Iconos con etiqueta accesible, contraste AA y tacto mínimo 48 dp. Sin degradados, colores por tarjeta ni tablas ilegibles.

Catálogo en tarjetas; pedidos e ingresos en listas compactas o tablas adaptables, detalle para campos extensos. Barra y navegación de A; no crear un segundo menú. Puede usarse acceso fijo al carrito si no tapa contenido ni teclado; no es obligación un footer idéntico al documento previo. Mostrar Carga, Vacío, Error y Reintento; conservar filtros al volver del detalle, cancelar respuestas obsoletas de búsqueda y paginar en servidor.

4 Pantallas de catálogo y compra

B1 Catálogo y detalle de producto

Ruta /tienda para Cliente. GET /api/productos con page, pageSize, busqueda y categoriaId. GET /api/categorias/catalogo para categorías. No usar proveedorId como filtro de Cliente porque no tiene acceso al catálogo de proveedores. Listado con imagen/placeholder, nombre, precio y disponibilidad. Paginación o «Cargar más» según hasNext; no asumir que primera página contiene todos los productos.

Stock cero: mostrar «Sin stock» y deshabilitar agregar. Stock positivo: permitir ver detalle y elegir unidades, indicando que se confirma al procesar pedido. El servidor vuelve a validar; no reservar stock en el cliente. Resolver imagenUrl relativa contra origen API, fallback si el archivo falla, texto escapado sin HTML arbitrario.

Ruta /tienda/productos/{id}: GET /api/productos/{id}, foto, descripción, categoría, precio y stock. No consultar /movimientos con Cliente ni construir ranking global con pedidos propios. La consigna no exige más vendidos: no hay requisito pendiente por esa ausencia.

B2 Carrito editable

Ruta /compras/carrito. Recuperar del servidor un borrador del Cliente y mantener su id seleccionado por sesión. GET /api/salidas está acotado al propietario; incluye bajas lógicas para él. Identificar borrador por eliminado=false, confirmarXCliente=false y confirmarXEmpleado=false; no existe /carrito/actual ni garantía de un solo borrador. Recorrer páginas necesarias y si hay varios permitir seleccionar cuál continuar. Guardar id no sustituye validación de acceso de servidor.

No crear una cabecera por visitar el catálogo. Al primer agregado crear POST /api/salidas con {observaciones}, luego POST /api/salidas/{id}/detalles con {productoId,cantidad}. Son dos solicitudes: si la segunda falla, conservar borrador y mostrar error parcial, sin crear otra cabecera en el reintento.

Líneas con nombre o fallback Producto #ID, precioUnitario de respuesta, cantidad, subtotal y quitar. Cambiar mediante PUT /api/salidas/{id}/detalles/{detalleId} con {cantidad}; cantidad=0 elimina. También existe DELETE de esa ruta para quitar explícitamente. POST repetido de producto sigue semántica del servicio actual: no asumir acumulación; refrescar línea/cabecera después. GET de cabecera y GET de detalle para revalidar al editar.

Total y descuentos vienen del backend. No mandar precio, subtotal o total para forzar importes. Evitar POST por cada pulsación de +/−; edición local y Guardar cantidad, bloqueo durante envío y manejo de 409. Cancelar pedido: DELETE /api/salidas/{id}, Cliente propietario y solo antes de aceptación operativa; confirmación explícita de baja lógica.

B3 Pago simulado obligatorio

Ruta /compras/pago/{salidaId}, solo Cliente propietario de borrador válido y no vacío. Mostrar líneas y total fresco obtenido de GET /api/salidas/{id}, resumen y aviso permanente «Simulación académica sin cobro real». Elegir método ficticio, por ejemplo tarjeta de prueba o billetera simulada, sin solicitar PAN, CVV, credenciales bancarias ni datos reales.

Simulación local simple: acción «Simular pago aprobado» y en pruebas opción «Simular pago rechazado». No librerías bancarias ni llamadas de cobro. Un rechazo o cancelación conserva el borrador y no envía confirmar-cliente. Una aprobación local permite confirmar el envío; diálogo final explica que queda pendiente de aceptación del negocio y que todavía no se descuenta stock.

Solo tras ese paso llamar POST /api/salidas/{id}/confirmar-cliente. Ante éxito, mostrar comprobante de demostración con id real, fecha, importe devuelto y texto «Pago simulado aprobado; pedido enviado». La API no persiste un estado bancario de pago: no inventar campo pagado, número de transacción real o garantía de comprobante persistente. Separar «simulación local aprobada» de «pedido enviado en servidor».

Ante error de envío, no mostrar compra completada. Ante timeout de confirmación, consultar el pedido: si confirmarXCliente=true, mostrar enviado; si no, permitir reintento explícito tras resolver estado. Deshabilitar doble toque. No volver a ejecutar una compra real: todo es una demostración sin dinero. Si se necesita persistir pago/referencia de pago en backend para otro requisito futuro, comunicar la carencia y no alterar esquema por cuenta propia.

B4 Mis pedidos y seguimiento

Ruta /compras/pedidos y detalle /compras/pedidos/{id}. GET /api/salidas paginado y GET por id, siempre con token Cliente. Estado derivado en orden: eliminado → Cancelado; confirmarXEmpleado → Aceptado; confirmarXCliente → Enviado, pendiente de aceptación; resto → Borrador. No inventar estados Entregado, Cobrado o Reembolsado.

Borrador permite continuar compra/cancelar; enviado permite solo consultar y cancelar mientras no fue aceptado; aceptado y cancelado son de solo lectura. La consulta de propietario incluye cancelados: mostrarlos en historial. Si la aceptación ocurre mientras intenta cancelar, tratar 409 y recargar. No permitir cambiar cantidades tras enviar. Comprobar que un Cliente no pueda leer ni modificar pedidos de otro alterando id o URL.

5 Pantallas operativas de inventario

B5 Productos

Ruta /inventario/productos para Personal (Administrador o Empleado). GET /api/productos paginado, filtros busqueda/categoriaId/proveedorId; este último refleja ingresos relacionados, no proveedor fijo de producto. Formulario con nombre hasta 150, descripción hasta 1000, precio no negativo hasta dos decimales y categoriaId válido. POST /api/productos, GET/PUT/DELETE /api/productos/{id}; no enviar stock ni imagenUrl en escritura. El stock inicial es cero y solo cambia con movimientos.

Selector de categoría por /api/categorias/catalogo; proveedores por /api/proveedores/catalogo, permitidos a Personal. Baja lógica con confirmación y refresco después de éxito. No prometer borrado en cascada de historia.

Imagen: POST /api/productos/{id}/imagen multipart campo File, JPG/JPEG, PNG o WebP hasta 5 MiB y MIME correspondiente; backend valida contenido real. Vista previa, cancelación sin solicitud y manejo de 413. Crear producto y subir foto son operaciones diferentes: informar creación exitosa aunque foto falle; reintentar solo foto. No existe endpoint de borrar imagen.

Detalle /inventario/productos/{id}: producto y GET /api/productos/{id}/movimientos?page=1&pageSize=10. Tipo, fecha, cantidad con signo, contraparte y precio histórico. Salidas ya tienen cantidad negativa: no invertirla nuevamente. No usar ese endpoint con Cliente. Auditoría de A cubre bajas/reversiones; la consulta operativa no sustituye toda la auditoría.

B6 Proveedores

Ruta /inventario/proveedores para Personal. GET/POST /api/proveedores y GET/PUT/DELETE /api/proveedores/{id}. Campos nombre requerido hasta 150, cuit hasta 20, email opcional válido hasta 150, telefono hasta 50 y direccion hasta 300. GET /api/proveedores/catalogo para selectores. Conflicto de CUIT/email se muestra como validación comercial, no caída de red. No imponer reglas fiscales ajenas al servidor.

Se necesita proveedor para registrar entradas. Esta vista permite preparar datos sin recurrir a Swagger, pero no hace falta un dashboard propio ni una tabla con todos los campos simultáneos. A enlaza a la misma vista desde administración, no crea otra.

B7 Entradas de mercadería

Ruta /inventario/ingresos. Personal consulta GET /api/ingresos con page/pageSize/productoId/proveedorId/desde/hasta. Solo Empleado puede crear y modificar. Administrador no hereda esos permisos aunque vea el listado: respetar los atributos reales del backend.

Nuevo ingreso: POST /api/ingresos con {proveedorId,observaciones}, observaciones hasta 1000. Fecha y empleado los asigna servidor. Detalle GET /api/ingresos/{id}; cabecera, estado, proveedor, líneas, total y confirmación. Si el proveedor ya no está en catálogo, mostrar Proveedor #ID en lugar de perder la fila histórica.

Agregar POST /api/ingresos/{id}/detalles con {productoId,cantidad,descAplicado}; cantidad >=1, descuento entre 0 y 100 con dos decimales. GET/PUT/DELETE /api/ingresos/{id}/detalles/{detalleId}; PUT contiene cantidad y descuento. PrecioUnitario y subtotales son de backend, no inventar campo de costo de compra manual. Refrescar cabecera y no asumir cómo se acumulan productos repetidos.

POST /api/ingresos/{id}/confirmar con confirmación «Sumar unidades al stock». Sin detalles o con proveedor/producto no disponible puede fallar; no afirmar stock aumentado hasta éxito. Confirmado bloquea edición de líneas. Refrescar producto para demostrar el incremento.

DELETE /api/ingresos/{id}: borrador produce baja lógica; confirmado revierte unidades y puede fallar por stock insuficiente. Presentar «Revertir ingreso» y advertencia específica cuando está confirmado; no describirlo como simple eliminación visual. Solo Empleado. No ejecutar pruebas de reversión en datos reales sin autorización del propietario.

B8 Solicitudes y registro de venta

Ruta /inventario/solicitudes para Personal. GET /api/salidas paginado con productoId/clienteId/desde/hasta; no acepta filtro de estado. Mostrar id, fecha, clienteId o nombre disponible, estado, total y ver detalle. Un filtro local debe decir «En esta página»; no llamarlo bandeja global de pendientes si no se recorrieron todas las páginas.

Empleado no tiene acceso a /api/clientes/catalogo: usar Cliente #ID. Administrador sí puede consultar ese catálogo para resolver nombre. No usar tokens ajenos ni lanzar peticiones prohibidas a propósito. GET /api/salidas/{id} y /detalles/{detalleId} para consulta de líneas. Producto eliminado o inaccesible puede representarse por id sin destruir su historia.

Aceptar solo enviado, activo y aún no aceptado. POST /api/salidas/{id}/confirmar tras mostrar resumen e impacto en stock. Aceptación operativa registra la venta y descuenta existencias. Refrescar salida y productos. Stock insuficiente o concurrencia → 409, conservar mensaje y recargar, sin reintento automático. No hay DELETE o rechazo de salida para Personal: omitir acciones falsas. Cliente propietario conserva cancelación antes de aceptación.

6 Matriz rápida de conexiones

Prefijo /api. Las rutas enumeradas son las actuales de la copia revisada; cotejar DTOs antes de implementarlas.

Pantallas	Métodos y recursos	Permiso
Catálogo y detalle	GET /productos; GET /productos/{id}; GET /categorias/catalogo	Todos
Producto operativo	POST /productos; PUT/DELETE /productos/{id}; POST /productos/{id}/imagen; GET /productos/{id}/movimientos	Personal

Pantallas	Métodos y recursos	Permiso
Proveedores	GET/POST /proveedores; GET/PUT/DELETE /proveedores/{id}; GET /proveedores/catalogo	Personal
Ingresos consulta	GET /ingresos; GET /ingresos/{id}; GET /ingresos/{id}/detalles/{detalleId}	Personal
Ingresos modificación	POST /ingresos; POST /ingresos/{id}/detalles; PUT/DELETE /ingresos/{id}/detalles/{detalleId}; POST /ingresos/{id}/confirmar; DELETE /ingresos/{id}	Empleado
Pedidos consulta	GET /salidas; GET /salidas/{id}; GET /salidas/{id}/detalles/{detalleId}	Personal o Cliente con alcance propio
Carrito y envío	POST /salidas; POST /salidas/{id}/detalles; PUT/DELETE /salidas/{id}/detalles/{detalleId}; POST /salidas/{id}/confirmar-cliente; DELETE /salidas/{id}	Cliente propietario
Aceptar venta	POST /salidas/{id}/confirmar	Personal
Nombres de clientes	GET /clientes/catalogo	Solo Administrador
Pago simulado	Lógica UI local y luego confirmar-cliente existente	Cliente; no endpoint bancario

No crear pantallas extra solo para usar todos los endpoints. Login, gestión de cuentas/categorías, auditoría, PDF/correo y logger son de A. B aporta contexto de errores al logger común, no un endpoint /logs inventado ni otra implementación de Serilog.

7 Errores fechas y conectividad

400/422: mostrar validación por campo o resumen; 401: delegar cierre de sesión; 403: permiso insuficiente sin borrar sesión válida; 404: dato no disponible y refrescar; 409: estado/stock cambió; 413: imagen excedida; 429: respetar espera. Nunca transformar error en colección vacía. En timeout de escritura, consultar efecto antes de reintentar. Deshabilitar doble toque y distinguir error parcial en operaciones compuestas.

Operativos desde/hasta son DateTime con comparación sobre creación; reportes de A usan DateOnly y criterio propio. Acordar UTC y probar bordes al convertir fechas. UI en es-AR, precios decimal y cantidades enteras; moneda no se infiere de un campo inexistente.

En Android localhost es el dispositivo. A configura URL: emulador estándar suele usar 10.0.2.2:5087 y teléfono físico IP LAN de PC accesible. B consume ApiConfig y resuelve imágenes contra API. No conectarse a SQL ni abrir puertos/cambiar certificados para sortear un fallo. HTTP de desarrollo no autoriza tráfico inseguro en Release ni desactivar TLS global.

8 Pruebas y evidencia académica

1. Catálogo paginado, filtros y producto sin stock; imagen ausente/caída no rompe UI.

2. Carrito con dos productos, cambio de cantidad, eliminación de línea, varios borradores y total servidor. Verificar semántica de cantidad cero.

3. Simulación rechazada/cancelada no confirma pedido ni descuenta stock. Simulación aprobada seguida de confirmación exitosa produce comprobante de demostración, sin fingir cobro real persistido.

4. Caso completo en entorno autorizado: producto stock 0; ingreso de 10 confirmado → 10; carrito de 2 y pago simulado/envío → sigue 10; aceptación operativa → 8. Verificar Mis pedidos, movimientos, auditoría y PDF con A.

5. Cliente diferente no puede ver o alterar pedido ajeno; Empleado no accede a auditoría/cuentas; Administrador no crea ingresos. Ocultamiento del botón y rechazo servidor se prueban por separado.

6. Cancelación antes de aceptación y bloqueo después; stock cambia entre carrito y aceptación; 409 sin inventario negativo; doble toque sin duplicados; timeout consultado antes de repetir.

7. Ingreso confirmado bloquea edición; reversión en datos de prueba cambia stock cuando procede y falla si stock insuficiente. No usar la base actual sin permiso de pruebas.

8. Registrar un fallo controlado cliente mediante AppErrorLogger sin datos sensibles. A verifica log técnico servidor; ambos revisan auditoría comercial real. Main incorpora GET /api/auditoria/acciones y GET /api/auditoria/acciones/{id}, solo para Administrador y con DTO {items,total,page,pageSize}. B no consulta esas rutas con Empleado ni Cliente: A verifica eventos después de las operaciones de B.

9. Ejecutar en emulador o teléfono Android: anotar dispositivo, API Android, SDK, revisión y pasos. Capturas/video de catálogo → carrito → simulación → seguimiento y operación de stock. Probar teclado, Atrás, safe areas y conectividad. Swagger/navegador PC no acredita la prueba móvil.

Tests automatizados de modelos de pantalla/servicios para total de respuesta, transiciones de UI, cancelación de simulación y errores; no duplicar internamente la lógica de stock del servidor. Integración HTTP real en entorno autorizado y pruebas manuales Android para WebView, selección de imagen y navegación.

9 Integración con la nueva auditoría del servidor

La rama auditoria ya está integrada en main por PR #28: https://github.com/LuAlzuri/TPN-5-Movil-Correcion/pull/28. No fusionarla otra vez ni duplicar el registro desde Android. La revisión estática identifica eventos en Confirmar/Ingreso, Revertir/Ingreso, Eliminar/Ingreso borrador y Confirmar/Salida. La simulación de pago, el envío del cliente y los demás CRUD no generan necesariamente entradas en esta bitácora; no inventarlas para que la demo parezca completa.

Para la prueba integrada, Empleado confirma ingreso y Personal acepta salida; después Administrador verifica las acciones mediante el módulo de A. Comparar entidadId con el id de cabecera y actor con quien realmente confirmó, distinguiéndolo del creador. Probar que un conflicto que revierte la transacción no aparezca como confirmación exitosa. La UI comunica éxito solo al terminar la solicitud de negocio, sin un segundo POST de auditoría.

Existe migración adicional 20261028172144_AuditoriaAcciones, número 15. Exigir entorno preparado antes de operar con el backend actualizado: una tabla faltante puede hacer fallar la confirmación comercial. No diagnosticarlo automáticamente como error de stock. Reportar 5xx y contexto al logger, sin filtrar detalles internos al usuario. A/propietario validan y autorizan la preparación de base; B no aplica migraciones ni cambia conexiones. El factory de diseño nuevo usa una base LocalDB fija y requiere especial cuidado al elegir explícitamente el destino de database update.

Esta auditoría complementa las vistas históricas de entidades y no reemplaza el registro técnico de errores. Las pruebas de días previos con la secuencia de 14 migraciones no certifican el backend actual. Actualizar la evidencia de integración antes de dar el módulo por terminado.

10 Entregables y paralelo

Hito 1: DTOs Comercio, clientes y páginas de lectura mientras A entrega infraestructura. Hito 2: productos/proveedores/ingresos y confirmación. Hito 3: carrito y pago simulado, pedidos y aceptación compartida. Hito 4: integración Android y evidencia junto con reportes, auditoría y logs de A. No esperar al final para probar red o roles.

Entregar docs/tp6/agente-2/INTEGRACION.md después de pantallas, con ruta UI, rol, método/URL, filtros, DTO y ejemplo sin secretos, respuestas, errores, confirmación e impacto de stock. Registrar qué se simula y qué se persiste realmente. Documentar limitaciones observadas sin convertir extras descartados en incumplimientos docentes.

Aportar a A un análisis de claves y dependencias de detalles de ingreso/salida, unicidad de líneas y precio/descuento históricos para NORMALIZACION.md. No concluir 3FN porque hay ids artificiales ni cambiar esquema desde móvil. A concentra el documento de normalización y B aporta evidencias de dominio, sin editar el mismo archivo.

Trabajo de dos alumnos y Git: propiedad de carpetas, contribuciones identificables en commits incorporados por el propietario, revisión conjunta e integración antes de entregar. Cada uno escribe pruebas/documentos propios; el dueño del host integra configuración. No publicación automática. Criterio final: compra y entradas reales contra API, simulación claramente rotulada, stock correcto y aplicación probada en Android, más evidencia completa del resto de la consigna a cargo de A.